

RESEARCH ARTICLE

Enhancing Cloud Job Failure Prediction With a Novel Multilayer Voting-Based Framework

AHMED ELKARADAWY^{1,2}, AYMAN ELSHENAWY³, AND HANY HARB¹¹Systems and Computers Engineering Department, Al-Azhar University, Nasr City, Cairo 11884, Egypt²National Telecommunications Regulatory Authority (NTRA), Giza 12577, Egypt³Software Engineering Department, Faculty of Information Technology, Al-Ahliyya Amman University, Amman 19328, Jordan

Corresponding author: Ahmed Elkaradawy (aelkaradawy@tra.gov.eg)

ABSTRACT In modern cloud data centers, accurately predicting job failures before they occur is essential for ensuring system reliability, availability, and efficiency. To address this challenge, researchers have progressively developed machine learning and deep learning techniques that examine cloud logs to identify patterns linked to such failures. To surpass the prediction accuracy of earlier models, this paper presents the Multilayer Multi-Prediction Framework (MMPF), a novel approach specifically designed to enhance failure prediction accuracy. The framework is divided into two distinct layers. The first layer aims to detect job failures using a fine-tuned voting mechanism, while the second layer identifies the specific failure type. In the first layer, several classifiers are integrated to build a robust predictive model, which outperforms conventional methods that rely on a single classifier. By aggregating the predictions from multiple base classifiers and making the final decision based on a weighted average of the predicted probabilities, this approach delivers significantly higher accuracy in failure prediction. Decision Trees, K-Nearest Neighbors, Extreme Gradient Boosting, Adaptive Boosting, and Artificial Neural Networks are the classifiers implemented in this layer. Once the failed jobs are detected, the second layer takes over to determine the failure type employing the Random Forest algorithm. Based on experiments conducted with the Google Cluster 2019 trace dataset, the framework achieved impressive accuracy: 99.83% in detecting failed jobs and 99.97% in identifying the nature of the failure.

INDEX TERMS Cloud computing, ensemble learning, failure prediction, machine learning.

I. INTRODUCTION

Cloud computing has changed the way businesses and consumers access and utilize computing resources [1]. By offering scalable and on-demand services over the Internet, this technology has become a cornerstone of modern IT infrastructure [2], [3]. Users and organizations of all sizes rely on cloud platforms to deploy applications, store data, and perform complex computations with exceptional flexibility, reliability, and efficiency [2], [4]. However, as cloud environments continue to evolve and expand, they present many new challenges in managing resources and keeping overall system reliability [1], [2], [5]. This aspect is critical and indispensable for maintaining customer satisfaction and meeting performance expectations in the cloud [1], [4]. Ensuring the

high availability and reliability of cloud services is a primary responsibility of cloud service providers (CSPs), as it directly impacts the quality of service (QoS) they provide to their customers [2], [3].

Failures in cloud jobs or services can happen because of software bugs, hardware malfunctions, or insufficient resources [3], [6].

When jobs fail during execution, they can lower the quality of service and negatively affect how users experience the system. These failures waste valuable resources like network bandwidth, central processing unit (CPU), memory, and disk space [1], [2], [6]. For instance, on December 14, 2020, several Google services, including Google Drive, Docs, Calendar, Play, YouTube, and Gmail, became unavailable for almost an hour due to infrastructure problems, according to the Google Cloud status dashboard. In mid-October 2022, users experienced an outage that affected Google Maps and

The associate editor coordinating the review of this manuscript and approving it for publication was Nitin Gupta ^{ID}.

Street View. Many had trouble loading images, and the Street View feature didn't work properly for some time [1], [2], [5], [7]. As cloud systems grow more complex, being able to spot potential failures before they happen is becoming more important than ever for keeping services reliable and consistent for users [8], [9]. Traditional techniques for failure prediction usually rely on simple models or manual oversight, which may not effectively capture the complex patterns and various failure modes common in modern cloud environments [3], [10], [11].

To address these challenges, this study introduces the MMPF, which seeks to improve job failure prediction through the use of advanced ML and DL algorithms and subsequently identify the failure class. Unlike normal single classifiers, this approach incorporates multiple classifiers, each contributing its specific perspective to the prediction process [9], [11], [12]. During the first stage, the model combines Decision Tree (DT), K-Nearest Neighbors (KNN), Extreme Gradient Boosting (XGBoost), Adaptive Boosting (AdaBoost), and Artificial Neural Networks (ANN) to enhance the accuracy and reliability of fault prediction. The final classification is made through soft voting. Meanwhile, the second-layer model focuses on predicting the type of failure for more effective handling. The concept behind merging various hybrid algorithms into a single voting classifier is to leverage the unique advantages of each model while reducing their limitations [13]. This approach proved highly effective, as demonstrated by the outstanding accuracy achieved in this study. By applying ML and DL algorithms, businesses can forecast equipment failures before they occur [12], [14], [15]. This approach helps companies boost their operations by minimizing unexpected maintenance and allowing them to plan maintenance tasks during scheduled downtimes.

The key contributions of this research are outlined as follows:

- The Google Cluster Trace 2019 dataset is studied to find the key characteristics and patterns linked to job failures on cloud platforms.
- This work introduces a multi-classifier approach to strengthen the system's ability to detect job failures with greater precision. By drawing on the unique advantages of multiple base models, it delivers more dependable and accurate results.
- To support this, a complementary solution identifies the nature of each failure, helping ensure that every issue is handled with the most appropriate response.

The rest of this paper is organized as follows: Section II reviews the existing research that forms the foundation for the proposed framework. Section III explains the dataset used in this study and introduces essential concepts like voting classifiers, ML, and DL. Section IV describes the proposed framework in detail, breaking down its components and modules. This section also looks closely at the dataset to explore how the features are related to one another. Section V discusses the platform used to develop the model, the evaluation metrics, and the experiments conducted, along with the

results obtained. Section VI concludes the work and discusses possible directions for further study.

II. RELATED WORK

The main goal of this section is to explore and review recent studies related to the proposed research. Many of the reviewed studies have applied statistical, machine learning, and deep learning methods to Google Cloud datasets to predict job or task failures. This study is not limited to predicting failures, as is common in many other studies, but also extends its benefits to identifying the type of failure. These techniques aim to improve cloud computing service reliability by predicting failures before they occur, which helps to mitigate downtime and resource wastage [2], [11], [14], [15].

Cloud computing systems often experience failures. According to Vani and Sujatha [15], these problems arise because of the environment's complexity, which leads to decreased reliability and efficiency. They point out that traditional methods like checkpointing and data replication simply aren't enough to keep up with the demands of modern cloud infrastructures. To address this, they propose a machine learning-based approach configured to enhance fault tolerance by detecting system failures in advance. Their work began with an analysis of task failure patterns and operational data extracted from cluster logs. They applied CatBoost to detect key features, applied a genetic algorithm for feature selection, and trained a fine-tuned multilayer perceptron to perform failure prediction. The proposed approach was evaluated using the Google Cluster 2019 dataset and compared against several ML models, proving reliable in diagnosing operational issues in advance. The model stated high reliability with an overall accuracy of 99.31%, precision of 98.38%, recall of 99.62%, and F1-score of 98.47%.

Abualhaj et al. [16] noted that although security layers such as Intrusion Prevention Systems (IPS) reduce threats (e.g., DoS attacks), they also lead to 29 to 42% higher registration delays, drawing attention to the trade-off between security and system efficiency, an important factor when developing failure prediction methods. Similarly, Kolhar et al. [17] explored how cloud data auditing involves trade-offs between security, privacy, and performance, particularly when accounting for processing and communication overhead. These considerations show the growing need for more effective failure prediction methods, such as multilayer voting-based models, to help keep cloud systems reliable as security threats become increasingly complex.

Moni et al. [18] introduced the AI-based Cloud Failure Prevention Algorithm (ACFP), which combines ML and DL techniques to analyze data from Google clusters. The goal was to predict job failures more accurately using models such as CatBoost, LightGBM, Gradient Boosting, and Random Forest (RF). From the set of models tested, the most effective one at identifying failures was selected. It analyzed patterns in resource usage, event types, and scheduler behavior to detect problems before they escalated. Their results showed that

ACFP improved prediction accuracy and reduced the failure rate from 22.76% to 2.67%.

Alahmad et al. [19] implemented a proactive strategy for task scheduling in cloud data centers to reduce the risk of task failures. To support their strategy, they examined large-scale datasets from Trinity, Alibaba, and Google, which provided valuable insights into failure patterns within complex cloud environments. They used their analysis to develop a framework that can predict whether a task will be completed successfully or fail during execution. When a failure is detected, the system responds by applying the appropriate corrective actions. Their results indicated that ANN and Convolutional Neural Network (CNN) reported accuracies of 94% and 92%, respectively, when applied to the Google Cluster 2011 dataset. Furthermore, their strategy recovered up to 40% of tasks expected to fail, according to the Alibaba dataset. By applying corrective actions, they saved considerable resources such as CPU and RAM within the clusters. Soualhia et al. [20] dedicated their research to predicting task failures as a way to optimize cloud computing resource management. They dealt with some ML models, such as DT, boosting algorithms, and RF. Among these algorithms, RF performed best, with accuracy close to 95%, and both precision and recall also showed good results, around 97%.

Building on the importance of reliability in cloud systems, Jassas and Mahmoud [21] proposed a method for catching failures early. To find the most reliable technique, they evaluated several algorithms, including DT, Logistic Regression (LR), KNN, Naive Bayes (NB), RF, and Quadratic Discriminant Analysis (QDA). From the set of models considered, the DT and RF models showed acceptable results, reaching 95% recall and 98% precision. Vani and Sujatha [22] studied the challenge of enhancing fault tolerance in dynamic computing environments, which experience different workloads, configurations, and system states. These changes create additional difficulties for predicting cloud failures, especially when the distribution of failed and successful jobs is imbalanced, leading to inaccurate predictions of rare failure events. Because traditional machine learning models struggle to understand the complexity of cloud jobs, they often produce inaccurate predictions and higher error rates. To predict failures in such environments, the authors recommended applying one of the ensemble ML techniques. This method was assessed using the Google 2019 dataset, and the results showed an enhanced ability to predict job failures before they occurred. The proposed method reached a high accuracy rate of 98.92%.

In their statistical analysis of cloud cluster operations, Shetty et al. [23] identified clear differences between failed and successful tasks in resource usage. These differences informed the design of an XGBoost classifier, which was able to predict task failures with 92% precision and 94.8% recall.

Salanke and Kowshik [24] addressed the challenges caused by failures in cloud resources, clarifying how task failures can affect system performance, disrupt the services, and negatively impact the QoE. To overcome these issues, the

authors proposed a method for predicting these failures using one of the important DL techniques, Bidirectional Long Short-Term Memory (Bi-LSTM). The suggested model took advantage of the sequential nature of task execution data to better understand dependencies and improve the accuracy of failure predictions. Jomah and Aji [25] conducted an extensive evaluation of the Google 2019 Cloud dataset, focusing on terminated jobs and how workloads behave in large-scale clusters. Recognizing the value of workload analysis, the authors focused on how understanding these features can enhance resource use and system performance in cloud clusters. They studied workload behavior at the job level, including how frequently they terminated. The insights gained from this analysis help clarify how cloud environments function and support better resource management for user demands.

Using workload traces from Google's Borg cluster (2019), Tuns and Spataru [26] evaluated the performance of some ML techniques in predicting task failures in distributed systems. The authors pointed out that detecting job failures in these environments is vital for reducing service downtime and managing costs. In dynamic data centers, failures can lead to large financial and productivity losses. The authors explored the effectiveness of four ML algorithms—DT, RF, GBoost, and LR—using workload data from industrial clusters. Through this process, they developed a GBoost-based classifier that achieved an accuracy of around 84%. This research offers valuable steps toward enhancing the reliability of cloud services by detecting issues before they occur. As demonstrated in [27], adaptive neural networks (e.g., EANN) notably improve cloud load prediction and resource allocation, highlighting the potential of machine learning-driven approaches like our multilayer voting-based framework to increase failure prediction accuracy by dynamically adapting to workload patterns and system demands.

Liu et al. [28] investigated the predictive analysis of task failures in the early phases of cloud operations by grouping jobs with similar attributes and applying Fuzzy C-Means (FCM). Each cluster was considered an independent dataset for the training and evaluation of predictive models. They compared three algorithms—Support Vector Machine (SVM), Extreme Learning Machine (ELM), and LR—noting that SVM and ELM obtained an accuracy of 89.96%, whereas LR attained 80.93%. Gao et al. [29] employed a multi-layer Bi-LSTM model to predict failures at both the job and task levels. The extraction of static and dynamic features for training enabled the model to reach an accuracy of up to 93% for task failures and 87% for job failures. Rosa et al. [30] analyzed patterns of job failures in the cloud logs by examining job workloads and system attributes. To classify termination states, they developed three DL-based models, which achieved an accuracy of 94.4% for jobs and 76.8% for tasks. Islam and Manivannan [31] extracted characteristics associated with application failures in the dataset. They applied an LSTM model to estimate whether

TABLE 1. Comparison of key studies on ML-based failure prediction.

Refer-ences	Models used	Dataset used	Modeling Insights and Gaps	Best Results
[15] (2022)	Hyperparameter-tuned multilayer perceptron	• Google	• Used only a Single model to classify jobs as failed or successful.	Accuracy (Acc) = 99.31%, Precision (Pre) = 98.38%, Recall (Rec) = 99.62%, F1-score (F1) = 98.47%
[19] (2021)	ANN and CNN	• Trinity • Alibaba • Google	• Used two separate models to detect failures, and no failure classification was implemented	ANN: Acc= 94% and CNN: Acc= 92%
[20] (2020)	DT, boosting algorithms, and RF	• Custom Hadoop workloads	• Tested multiple algorithms individually • Failure classification was not applied	RF (Best): Acc = 94.5%, Pre = 97.4%, Rec = 96.5%
[21] (2020)	DT, LR, KNN, NB, RF, and QDA.	• Google • Mustang • Trinity	• Tested 6 ML models individually • No failure classification applied	DT (Best): Pre=98%, Rec= 95%, F1 =97%
[22] (2024)	XGBoost, AdaBoost, Gradient Boost, And Light GBM	• Google	• Applied 4 ML Algorithms individually • No failure classification applied	XGBoost (Best): Acc = 98.92%, pre = 97.38%, f1 = 97.92%, Rec = 98.64%
[24] (2024)	SVM, LSTM, and Bi-LSTM	• Google	• Introduced a dynamic task migration strategy and test environment: CloudSim	Bi-LSTM (Best): Acc = 93%
[26] (2023)	DT, RF, GBoost, and LR	• Google	• Applied 4 ML algorithms to predict failures, and only one model is used at a time • No failure classification applied	DT (Best): Acc= 98.79%, F1 = 98.8%, Rec = 98.85%
[29] (2020)	Bi-LSTM	• Google	• Applied one DL algorithm to predict failures • No failure classification applied	Acc= 93%, F1= 92%
[36] (2022)	LR and KNN	• Bit Brains dataset	• Tested 2 ML Algorithms individually • No failure classification applied • Test environment: Google Colab Pro+	KNN: Acc = 99%, Pre = 99% and LR: Acc = 95%, Pre= 97%
[37] (2019)	SVM, LDA, CART, RF, KNN	• NERSC	• No failure classification applied • Tested the models individually	SVM (Best): Acc = 90.76%, Rec = % 67
[38] (2022)	RF, DTs, NB, and QDA	• Google • Mustang • Trinity	• Applied ML models separately • No detection of failure types • Test environment: Google Colab Pro+	DT (Best) on Google Trace: Acc = 99%, Pre = 99%
[39] (2023)	Ensemble Model, ANN, and KNN	• Google	• Used an ensemble technique • No detection of failure types • Test environment: AWS SageMaker	Ensemble (Best): Acc = 99.13%, Rec = 99%, Pre = 97%, F1 = 99%

an application would terminate, reaching an accuracy as high as 87%.

Liu et al. [32] criticized offline prediction methods for cloud operations and proposed an Online Sequential Extreme Learning Machine (OS-ELM) for job termination prediction. OS-ELM achieved up to 93% accuracy. Chen et al. [33] investigated the key features associated with job and task failures in cloud computing. They studied job events and resource usage to identify failure-related features and used RNN for prediction, reaching an accuracy of approximately 84%. El-Sayed et al. [34] analyzed log data from major computing clusters, such as Hadoop, Google, and Trinity, to uncover the factors contributing to job and task failures. They applied

RF for failure identification, achieving recall and precision rates of up to 94% and 95%, respectively. Hongyan et al. [35] evaluated the effectiveness of multiple ML classifiers, such as Gradient Boosting Decision Tree (GBDT), RF, KNN, and LR, to predict job failures caused by misconfigurations, based on the OpenCloud dataset. GBDT recorded an accuracy as high as 92%, along with a precision of 78% and a recall of 52%. Table 1 gives an overview of some of the most relevant studies in this area, looking at the methods they used, the datasets they worked with, and how they measured performance. Looking through these studies, it's clear that there are still a few gaps. Most of them use only one classifier at a time, which doesn't always capture everything and can

lead to results that aren't entirely accurate in certain cases. One thing that really stands out is that none of the studies actually focus on identifying the specific type of failure that occurred. That's something we felt was missing and needed more attention.

III. BACKGROUND

A. GOOGLE CLUSTER DATASET 2019 DESCRIPTION

The Google dataset covers cloud resource usage across eight different clusters. Each of these clusters had around 12,000 nodes or servers as of May 2019, and the dataset was released in early 2020 [18], [19], [20], [35]. This dataset focuses solely on resource requests and usage, as described in Table 2 [3], [35], [36], [40]. It was created by recording tasks and jobs submitted during May 2019 and has a total size of about 2.4 TB [3], [19], [25], [31]. Each entry in the dataset corresponds to a pool of machines within a specific cluster, which is responsible for distributing and managing work among the machines [18], [19], [37], [41].

A Google cluster contains multiple physical machines organized into physical racks and communicated via a high-bandwidth network within the cluster [38], [39], [42], [43]. Within a cluster, a "cell" refers to a set of machines that operate under a unified cluster-management technique, which coordinates the allocation of tasks to these machines. In this system, there are two main types of resource requests: *jobs* and *alloc sets*. A job, which consists of one or more tasks, represents a computational unit that a user wants to execute. An "alloc set," on the other hand, comprises one or more "alloc instances" and is used to reserve resources where jobs can be executed [19], [22].

Each task is essentially a Linux program, potentially incorporating concurrent processes, that runs on a single machine [3], [25], [32], [41].

TABLE 2. Google 2019 traces.

Date	May 2019
Duration	31 days
Cells (clusters)	8
Machines	96.4k
Machines per cell	12.0k
Hardware platforms	7
Machine shapes	21
Priority values	0–450
Alloc sets	Y
Job dependencies	Y
Batch queueing	Y
Vertical scaling	Y
Format	BigQuery tables

When a job is assigned, it may specify an "alloc set" for its execution. In this case, each task within the job will use resources from one of the "alloc instances" within that set. If a job does not specify an alloc set, its tasks will

directly use resources from the machine on which they run. Figure 1 illustrates all possible scenarios that can arise during the lifecycle of a job [44], [45]. In this context, the terms "collections" refer to jobs and *alloc sets*, while "instances" describe tasks and *alloc instances*. Due to the large size of the dataset, a representative sample was chosen for analysis. This sampling approach allowed for more efficient data handling and processing [3], [44], [46], [47].

A collection or instance ends through any of the following five causes:

- Finish: The task was completed successfully.
- Evict: The task was removed due to various reasons, including:
 - Hardware failure.
 - A forced OS upgrade.
 - Preemption by a higher-priority task.
 - The actual demand exceeded the machine's capacity, causing the scheduler to kill instances to free up resources [3], [19], [46].
- Kill: The user canceled the task, either directly or due to the termination of a parent task.
- Fail: The job or task was unexpectedly terminated due to an internal issue, such as a segmentation fault or exceeding the allocated resources (e.g., caused by a memory leak or misconfiguration) [3], [47].
- LOST: Describes cases where a termination was expected, but the data indicated that it was missing [3], [46], [48].

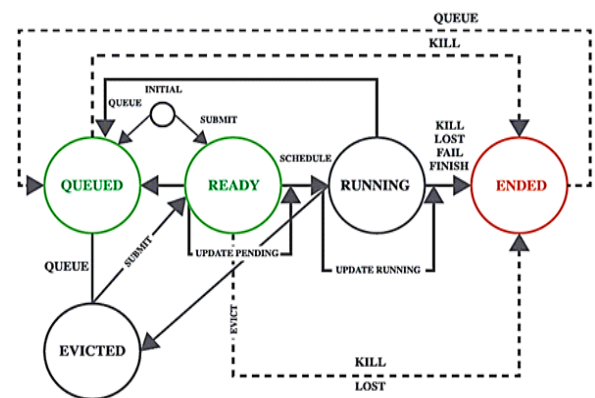


FIGURE 1. [3] Job lifecycle diagram.

B. VOTING CLASSIFIERS

Many researchers have used single traditional ML or DL methods to predict job or task termination status on the Google dataset [26], [38], [45], [48]. Traditional ML algorithms have a diverse range of techniques, including SVM, HMM, DT, LR, RF, KNN, AdaBoost, XGBoost, and others [15], [21], [49]. On the other hand, the widely used DL algorithms are ANN, CNN, LSTM, Bi-LSTM, and GRU [19], [41], [50]. Integrating multiple algorithms into a voting classifier model enhances its predictive capabilities and makes it more beneficial to CSPs for practical implementations

in cloud computing environments when compared to single classifiers [26], [51], [52]. Additionally, mixing ML and DL algorithms in the base layer of the voting classifier offers many significant advantages and enhances the overall performance of the proposed approach. ML techniques often operate under the assumption that data segments are independent, which makes them less suitable for most datasets, where segments exhibit interdependence and correlation [26], [53]. Therefore, DL methods such as ANN can handle extended time-series data and discover the relationships among data segments within the dataset [19], [51], [54]. Voting classifiers are powerful ensemble techniques that combine the outputs of multiple classifiers to improve model accuracy [13], [32], [55]. They aggregate predictions from several models to make a final decision [47], [50], [56]. These classifiers can be categorized into hard voting (majority voting) and soft voting (weighted averaging of probabilities) [20], [50], [51].

The first-layer model in the planned framework focuses on using the best features of various combined ML and DL algorithms to achieve better accuracy and robustness compared to the single classifiers commonly used in related studies. Combining predictions from different techniques reduces the possibility of overfitting, enables the model to perform well on unseen data, and overcomes the drawbacks of relying on single classifiers [20], [41], [57]. The hard-voting method selects the final class label based on the majority decision among the participating classifiers. For example, if three classifiers produce predictions of classes M, N, and M, the voting mechanism will select class M as the final output. On the other hand, soft voting works by averaging the predicted probabilities from multiple classifiers and selecting the class with the highest mean score. Unlike hard voting, which counts decisions equally, soft voting values the level of confidence each model has in its prediction. When one model predicts a class with 95% confidence and the others are less sure, that confident prediction usually carries more weight in deciding the outcome. By weighting predictions in this way, soft voting can improve the consistency and reliability of results, particularly when the accuracy of models varies [41], [47], [57].

C. MATHEMATICAL MODELLING OF THE FINE-TUNED VOTING CLASSIFIER

To compute the weighted average of predicted probabilities across all base models, we use Equation (1). For each class $c \in C$, where C is the set of all possible classes, the final probability is calculated based on the weighted outputs of all models.

$$P_{\text{final}}(c) = \frac{\sum_{i=1}^n w_i \cdot P_i(c)}{\sum_{i=1}^n w_i} \quad (1)$$

where:

- $P_i(c)$ is the predicted probability of class c from model i ,
- w_i is the assigned weight of model i ,
- n is the total number of base models.

Next, the model makes its final decision by choosing the class with the highest overall weighted probability, as shown in Equation (2).

$$\hat{y} = \arg \max_{c \in C} P_{\text{final}}(c) \quad (2)$$

where:

- $\hat{y}$ is the predicted class,
- $P_{\text{final}}(c)$ is the final weighted probability of the class c .

D. NEURAL NETWORKS

A deep neural network typically consists of a multilayer architecture where neurons are fully connected from one layer to another [6], [29], [58]. The basic architecture consists of an input layer, a single hidden layer, and an output layer. By adding more hidden layers, the network becomes deeper and more complicated [6], [59]. Each layer consists of some neurons (or nodes), and every neuron in one layer connects to all neurons in the following layer through synapses, allowing for complex data processing and feature extraction [59], [60]. ANNs are used in the proposed voting classifier with four other ML algorithms to capture complex patterns in the Google cluster dataset. Their flexibility and ability to model nonlinear relationships make ANNs fundamental to a large number of modern DL applications [4], [6], [19], [56].

E. ADVANCED MACHINE LEARNING

The other four ML algorithms used as base classifiers in the proposed voting classifier are DT, XGBoost, KNN, and AdaBoost. Each of these algorithms has unique characteristics.

In DTs, data are split into groups guided by the values of their input features. At each internal node, a decision is made based on a feature, and the leaf node shows the final outcome [20], [48], [49]. Creating a DT requires selecting the feature that most accurately partitions the data at each node. DTs are favored for their simplicity and interpretability, allowing users to visualize the decision-making process [51], [60], [61].

KNN works on the assumption that data points with similar features are positioned near each other in feature space. It assigns a class to a new data point by considering the 'K' nearest neighbors in the training data and selecting the most common class among them [48], [53], [55], [57]. XGBoost builds a set of DTs, with each one trained to correct the mistakes made by the previous tree, heading to steady improvements in accuracy [20], [23], [25]. A key feature of XGBoost is its use of techniques that help reduce overfitting and improve the model's ability to perform well on new data [59], [61].

AdaBoost is a robust ensemble approach that combines several basic classifiers to generate a strong predictive model [50], [55], [58]. The core idea behind AdaBoost is to focus on the errors made by previous classifiers, adjusting their weights to improve the final results. This method works well in situations where there are only two classes to predict [28], [58], [59]. In AdaBoost, each weak classifier

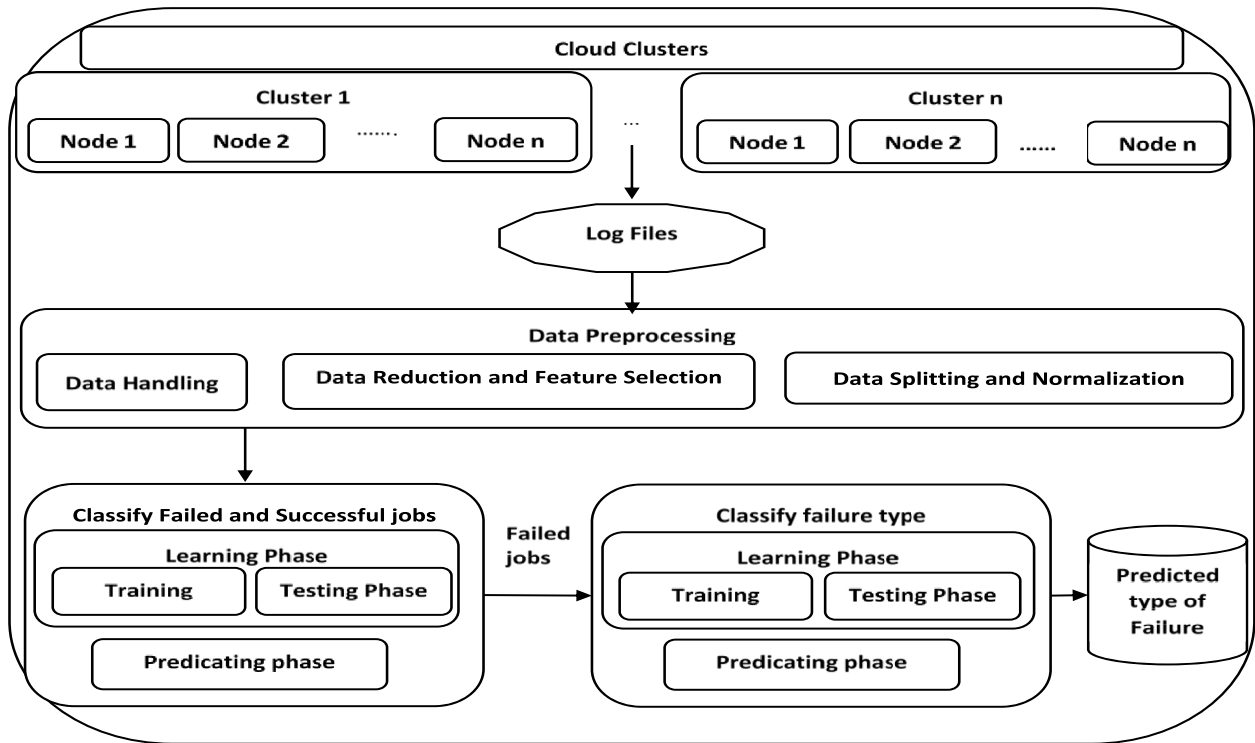


FIGURE 2. Multilayer, multi-prediction framework.

is trained one after the other, giving more attention to the instances that were misclassified in earlier rounds. The final prediction is based on combining the weighted contributions of all the weak classifiers' outputs [19], [60].

IV. THE PROPOSED FRAMEWORK

This study aimed to predict whether a job would succeed or fail, as well as to determine the type of failure if it occurred. Events such as “Evict,” “Fail,” “Lost,” and “Kill” were treated as failure outcomes, while “Finish” reflected successful results [3], [45].

A. MULTI-LAYER MULTI-PREDICTION FRAMEWORK

This section describes the proposed MMPF, which is designed to protect cloud services and applications. The objective of this framework is to predict the termination status of running jobs, whether they will be completed successfully or unsuccessfully, as early as possible during their execution time. This early prediction approach supports better resource management and ensures system availability. The second objective of the framework is to identify the type of failure. This approach enables more efficient management of each failure type by applying the most suitable response strategy. Figure 2 illustrates the overall framework, which consists of three key modules: Data Preprocessing, Job Failure Prediction, and Failure Type Detection.

Each of these modules is described in the following sections. Importantly, this framework is versatile and not tied to

any specific cloud vendor. It can be applied to any dataset that includes relevant information for predicting job termination status and identifying the types of failure.

This helps CSPs better manage and address these issues. However, the framework will need some configuration adjustments to fit each specific cluster platform.

1) DATA PREPROCESSING

Data preprocessing includes tasks such as data extraction, transformation, cleaning, feature selection, data splitting, and normalization/standardization. The *split_cell()* function helps to extract numerical data (e.g., CPU, memory) from structured strings and convert unprocessed data into a format appropriate for data analysis or model training. It also manages missing or non-numeric values and robustly handles non-string inputs. Data cleaning involves removing rows with missing values to ensure the dataset's completeness and consistency. Finally, the index is reset to maintain sequential indexing after row removal.

Feature engineering or selection focuses on improving how well ML and DL models perform by creating, selecting, or adjusting attributes. The heatmap in Figure 3 shows strong positive correlations between the memory-related features (assigned, requested, maximum usage) and CPU-related features (requested, maximum usage), highlighting effective resource planning.

Additionally, the positive correlation between *priority* and *alloc_collection_id* implies that higher-priority jobs are more

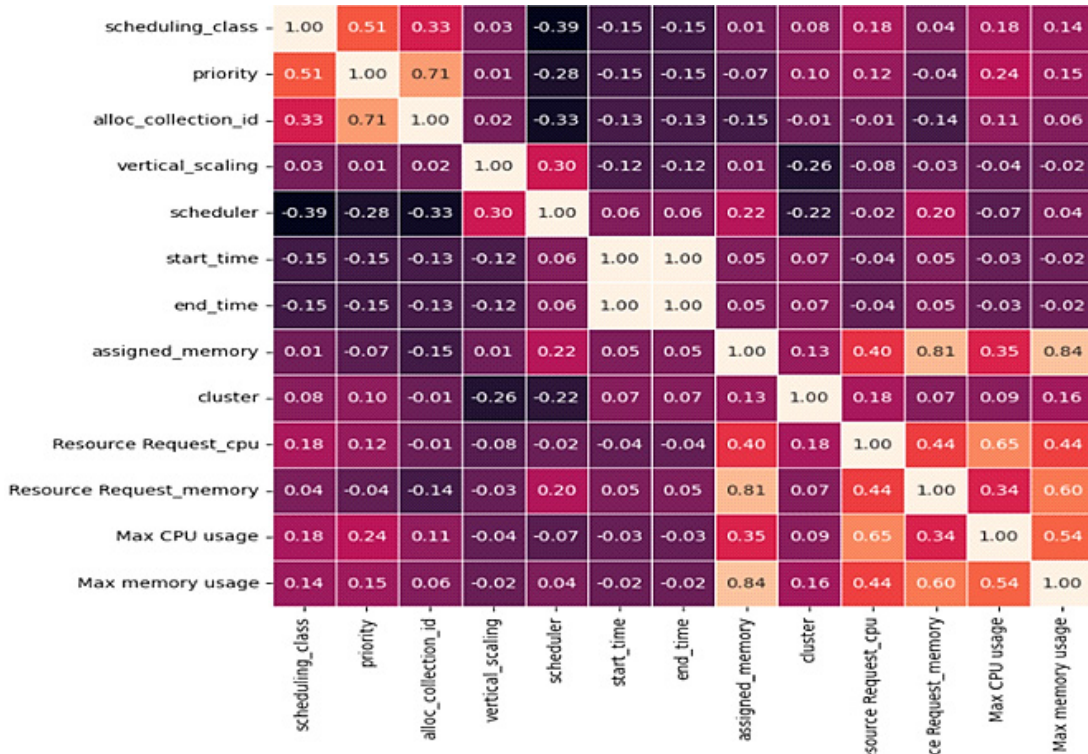


FIGURE 3. Correlation between features.

likely to use specific allocation strategies. In comparison, the *scheduling_class* feature has a negative correlation with both *priority* and *scheduler*, meaning that lower scheduling classes generally have higher priority.

Start and end times, along with *vertical scaling*, exhibit weak correlations, suggesting they have little impact on resource requests or usage. The *drop ()* function is used to remove specific columns that are not important from the data frame “df”, producing a new data frame “dff”. Attributes or features such as “user”, “tail_cpu_usage_distribution”, “resource_request”, “start_after_collection_ids”, and others are removed because they are not useful for model building. These steps help simplify the dataset by reducing its dimensionality and concentrating only on the relevant features necessary for constructing the target model. Next, the data is organized by separating the features (X) used for prediction from the labels (Y) being predicted. After that, the data is split into two separate sets: 70% is used for training the model, and the remaining 30% is set aside to test how well the model performs. One of the important steps in both ML and DL is feature scaling. It ensures that all input variables are on a similar scale and can improve the performance of certain ML algorithms, such as KNN, in the proposed model. Figure 4 outlines the main steps used to prepare the data before training.

2) JOB FAILURE PREDICTION METHOD

This model combines several ML and DL algorithms to build a multi-classifier system that handles various prediction

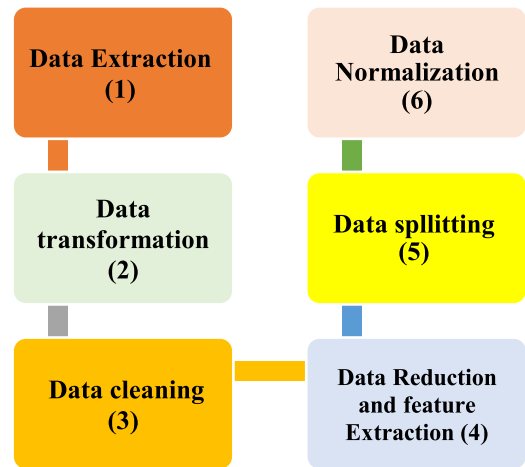


FIGURE 4. Data preprocessing steps.

tasks. DT, KNN, XGBoost, AdaBoost, and ANN are the five algorithms that are integrated into the model’s voting classifier. We used this ensemble technique to improve failure prediction in large cloud clusters through the classification of job termination statuses. The system is designed to be resilient, to handle noise and outliers in the dataset without difficulties. When a model makes an error, others in the ensemble can often detect and correct it. By combining multiple classifiers, this approach employs the unique strengths of each to enhance job failure detection using the Google Cluster Traces dataset.

3) TYPE OF FAILURE PREDICTION METHOD

In the above layer of our framework, we applied the RF algorithm to identify what kind of failure occurred. After finding that a job has failed, the RF then classifies the failure into one of four classes: Evict, Fail, Lost, or Kill. Understanding the type of failure helps CSPs take the right actions to fix the issue. By implementing this model, providers such as Alibaba, Amazon, and Microsoft could rapidly enhance their cloud performance and service quality.

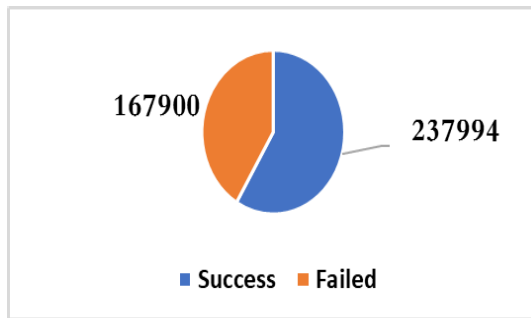


FIGURE 5. Success and failed jobs in the dataset.

TABLE 3. Failed and successful jobs across different clusters in the trace.

Cluster	Successful jobs	Failed jobs
1	21209	21504
2	24408	20009
3	36120	22663
4	29843	25659
5	35157	13500
6	31809	26722
7	30261	20099
8	29187	17744

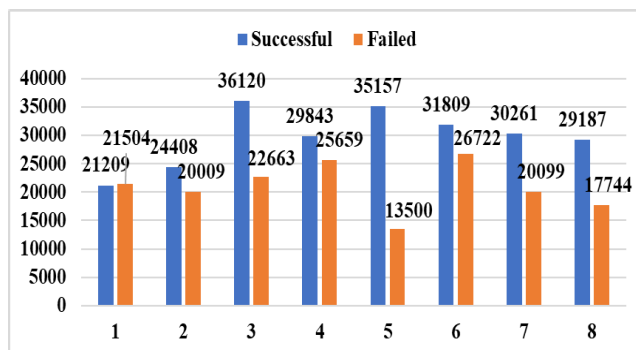


FIGURE 6. The distribution of both failed and successful jobs across eight Google clusters.

B. TRACES ANALYSIS

The dataset, which was created from eight tables with a total of 33 columns, provides useful insights that can help refine

modeling strategies and improve failure prediction methods. Although the dataset exhibits class imbalance, with 167,900 failed jobs and 237,994 successful jobs, the sheer volume of data minimizes the potential impact on model training. Figure 5 illustrates the imbalance in the data. However, the large sample size ensures there is sufficient data for reliable training and evaluation.

Table 3 provides information on both failed and successful jobs across eight Google clusters worldwide. Figure 6 shows how these jobs are distributed across the clusters. Each job is assigned to one of four scheduling classes (0, 1, 2, and 3), based on its sensitivity to latency.

Table 4 shows how the jobs are distributed among these classes. The distribution of all jobs across the scheduling classes serves as a critical indicator for evaluating how latency sensitivity impacts job performance and failure rates. Analyzing the job distribution across these classes can help identify which ones are more likely to experience success or failure.

TABLE 4. Distribution of successful and failed jobs across scheduling classes.

Scheduling Class	Successful Jobs	Failed Jobs
0	87014	33400
1	58493	46836
2	61963	69604
3	30524	18060

Figure 7 illustrates the relationship between scheduling classes and termination status (whether jobs failed or were completed correctly). While there is no obvious link between lower scheduling classes and job failure, the majority of jobs are found in the lower scheduling classes (0, 1, 2), with fewer jobs placed in the highest scheduling class (3). In scheduling class (2), the number of failed jobs surpasses the successful ones. Interestingly, a notable portion of jobs in the scheduling class (1) has also been marked as failing. This indicates a strong correlation between scheduling classes (1) and (2) with job failures.

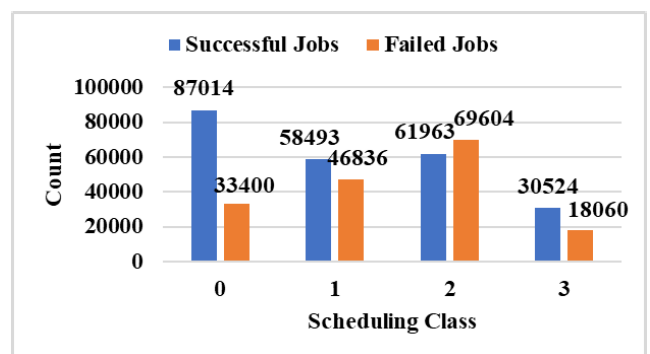


FIGURE 7. Assignment of successful and failed jobs to the four scheduling classes.

The observed correlation suggests that jobs in the highest priority class are not necessarily more successful. This could imply that the scheduling method or resource allocation strategy may not be optimized appropriately. To address this, it may be necessary to adjust the scheduling approach to better balance resource distribution and job priority. Identifying why certain classes tend to have higher failure rates can offer clear indicators to enhance the scheduling system, thereby increasing system performance and job completion rates.

Each task or job is given a priority level, represented by a numerical value from 1 to 450. A lower number indicates lower priority, and higher numbers mean higher priority. Jobs that fall at the top of the scale typically receive more attention and resources, while those ranked lower may have to wait or operate with limited support. This prioritization of tasks can guarantee that critical tasks are completed first, but it may also lead to resource constraints for lower-priority tasks. Analyzing how different priority levels affect outcomes can deliver valuable information for optimizing resource distribution and improving task management effectiveness.

V. EXPERIMENTS

The primary objective of these experiments is to evaluate the MMPF, which consists of five ML and DL models linked with a tuned-voting classifier in the bottom layer and the RF algorithm in the upper layer. Accuracy serves as our main evaluation metric, while other metric aspects, such as precision, recall, and F1 score, were calculated for a complete assessment. Evaluating the algorithms individually or as a group using these metrics gives us a solid understanding of their effectiveness. Recall helps highlight the model's ability to detect important issues, which is key when even a single error could have a big impact. To get a more balanced view, we look at the F1-score, which combines both recall and precision. Altogether, these metrics show that the proposed framework can handle different scenarios well, making it a strong option for use in cloud data centers.

A. KAGGLE PLATFORM

Our framework was developed and executed on Kaggle, a cloud-based platform that delivers different computing services and resources to the public. Similar to Jupyter Notebooks, Kaggle Notebooks provide an interactive space for coding in Python and R. Users can either explore Kaggle's public datasets or upload their own to analyze. The platform gives free access to powerful computing tools such as graphics processing units (GPUs), which are essential for running complex algorithms and conducting large-scale experiments. For model training and evaluation, the authors used Kaggle's NVIDIA Tesla T4 GPU, along with 40 GB of storage and up to 330 GB of RAM. The Tesla T4 GPU outperforms CPUs in training neural networks and running complex models because of its parallel processing architecture. Kaggle also offers other GPUs, including the K80 and P100, depending on availability. Each notebook instance includes local disk

space for temporary data storage, making it easy to save data and results.

B. EVALUATION OF THE BASE ALGORITHMS AND THE VOTING CLASSIFIER

1) EVALUATING THE JOB FAILURE PREDICTION MODEL

Instead of relying on default settings, which often do not perform well in practical situations, these experiments focused on carefully tuning hyperparameters. Each model was given its own set of key parameters organized in a dictionary, specifically chosen for fine-tuning. To manage this process, the experiments used GridSearchCV, a built-in tool from the Scikit-learn library. It explores every possible combination of parameters and uses three-fold cross-validation to ensure a fair and detailed evaluation. The objective was not only to achieve acceptable results but also to determine the most accurate and dependable configuration for each model. Once the best settings were found, the top-performing version of each model was saved for future use. After tuning the individual models, they were brought together into a voting ensemble. This involved testing both hard and soft voting strategies and adjusting the weights assigned to each classifier to achieve the most balanced and accurate overall result. We used GridSearchCV once more, this time to directly optimize the ensemble model and boost its performance. After tuning, all models are tested. Both the individual and combined models, to see how well they performed. We tracked the results closely and compared key metrics to assess how much the tuning process helped. The performance of the tuned classifiers was assessed, and their evaluation metrics are as follows:

- DT: This model produced perfect precision, recall, and F1-scores of 99.68%, leading to an overall accuracy of 99.68%. The model achieved its best performance when it was allowed to grow without any depth limits (`max_depth = None`). This setup enabled the model to split nodes until pure leaves were achieved fully. By setting `min_samples_split = 2`, the tree could divide even the smallest groups, allowing it to capture detailed patterns, but at the risk of overfitting to noise in the data.
- KNN: This classifier outperformed expectations, with precision, recall, and F1-scores all reaching 99.72%, which also led to an accuracy of 99.72%. The optimal configuration used three neighbors and applied a distance-based weighting strategy, giving greater importance to closer instances. This localized focus helped improve classification performance, particularly in regions with subtle decision boundaries.
- XGBoost: This model showed great performance, achieving 99.26% in both precision and accuracy. Its recall and F1 score were also notable, each at 99.26%. The model operated with a learning rate of 0.3, a maximum tree depth of 6, and 100 boosting rounds. The model used this setup to learn useful patterns in the data and still performed well on new, unseen inputs.

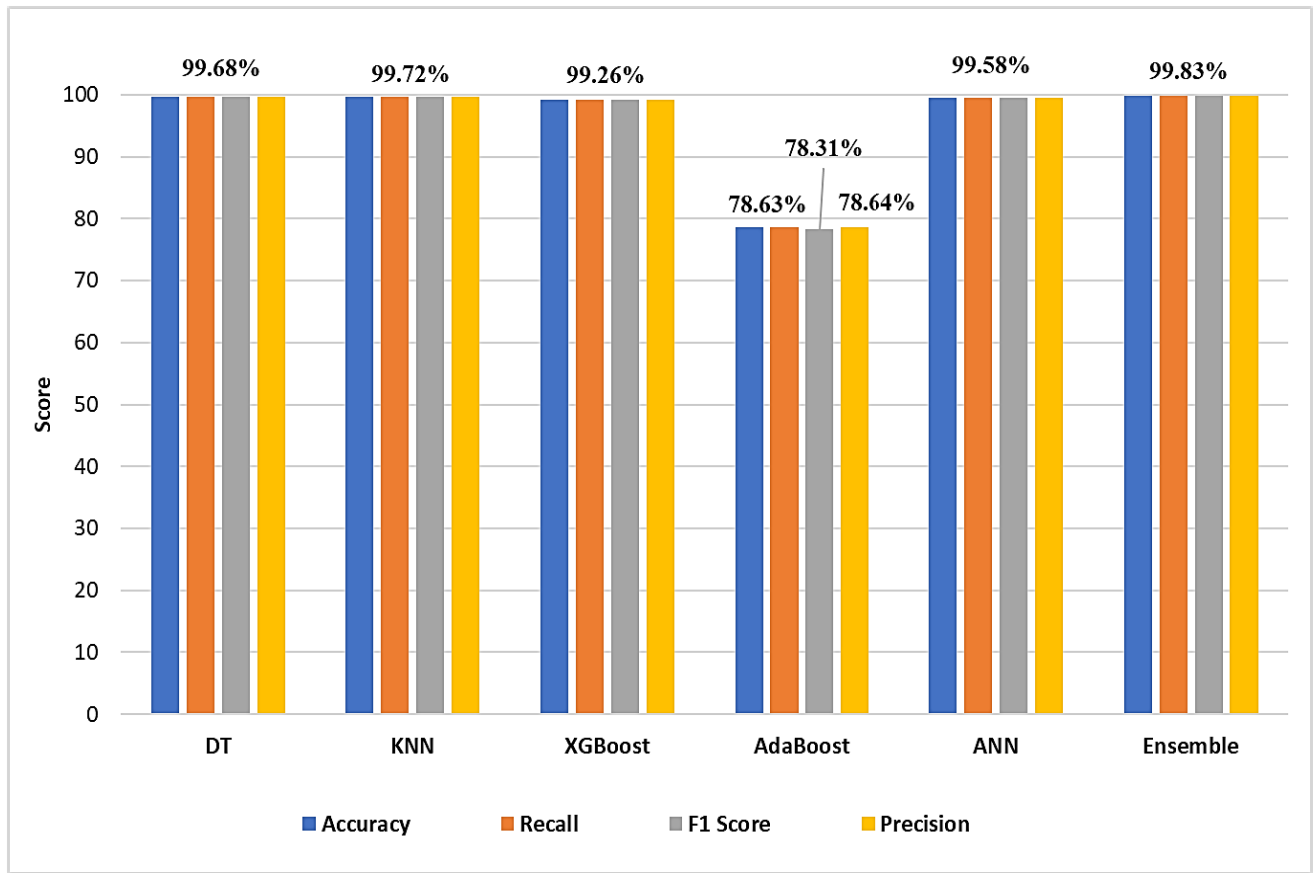


FIGURE 8. Performance metrics of tuned classifiers.

- AdaBoost: Among the models tested, it showed the lowest performance, with precision at 78.64%, recall at 78.63%, an F1-score of 78.31%, and overall accuracy at 78.63%. This suggests the model may have difficulty processing numerical data. By choosing a learning rate of 1.0 and incorporating 100 weak learners, AdaBoost adopted a more assertive training approach. With this configuration, the model balanced complexity and generalization, avoiding overfitting.
- ANN: The ANN classifier performed reliably, scoring precision, recall, and F1 scores of 99.58%, which matched its overall accuracy of 99.58%. It used a two-layer architecture with 264 and 128 hidden units, along with the tanh activation function. The training phase ran for 200 iterations using the Adam optimizer, which adaptively modified the learning rate. The model learned intricate data relationships under this setup and performed consistently on validation data.
- Voting Classifier: This ensemble-based classifier achieved 99.83% in precision, recall, F1-score, and overall accuracy, outperforming all individual models. When compared to single model approaches, ensemble learning effectively improves predictive accuracy, as illustrated in Table 5 and Figure 8. The Voting

Classifier performed best with soft voting. Instead of just counting model votes, it considered how confident each model was in its prediction. The final configuration assigned greater weights to KNN and XGBoost ([1, 2, 2, 1, 1]), reflecting their superior predictive contributions within the ensemble.

TABLE 5. Comparison between the results of tuned models.

Algorithm	Accuracy	Recall	F1 Score	Precision
DT	99.68	99.68	99.68	99.68
KNN	99.72	99.72	99.72	99.72
XGBoost	99.26	99.26	99.26	99.26
AdaBoost	78.63	78.63	78.31	78.64
ANN	99.58	99.58	99.58	99.58
Ensemble	99.83	99.83	99.83	99.83

- The process of fine-tuning the Voting Classifier turned out to be resource-intensive. Each model required careful tuning, and only after that could the ensemble be adjusted. When the ensemble includes models such as ANN and XGBoost, the total time and processing power required increase substantially.

2) EVALUATING THE TYPE OF FAILURE PREDICTION MODEL

The goal of this section is to evaluate the RF model, which predicts the type of failure. The possible failure outcomes are 'EVICT,' 'FAIL,' 'LOST,' and 'KILL.' The evaluation metrics show that the RF model has performed exceptionally well, with most predictions being accurate. Only a small number of samples were misclassified, resulting in an overall accuracy of 99.97%. Figure 9 presents the confusion matrix, with the actual labels on the rows and the predicted labels on the columns.

	EVICT	FAIL	LOST	KILL
EVICT	4510	0	0	0
FAIL	0	27845	3	0
LOST	0	6	17731	0
KILL	0	6	0	269
	EVICT	FAIL	LOST	KILL

FIGURE 9. RF confusion matrix.

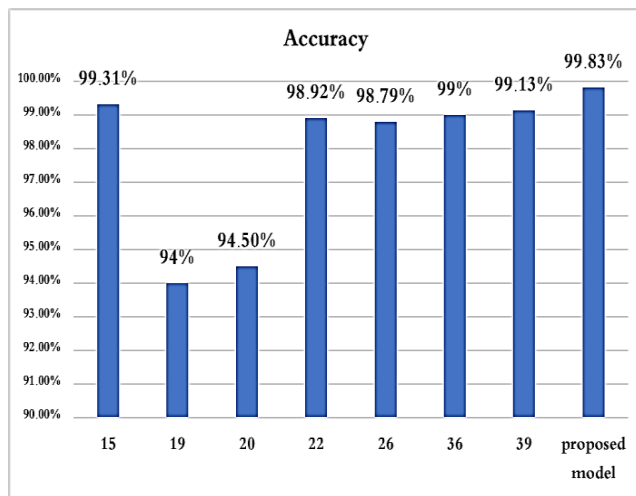


FIGURE 10. Accuracy benchmarking between the proposed model and reference models for failure prediction.

- Row 1 (Class EVICT or 0): All 4,510 samples were accurately classified, with no misclassifications into other classes.
- Row 2 (Class FAIL or 1): The model correctly identified 27,845 samples. Only 3 were mistakenly predicted as class 2, with no errors related to classes 0 or 3.
- Row 3 (Class LOST or 2): 6 samples from this class were wrongly classified as class 1, while 17,731 samples

were correctly identified. No further misclassifications occurred.

- Row 4 (Class KILL or 3): The model successfully classified 269 samples, with 6 misclassified as class 1. There were no mistakes in classes 0 or 2.

VI. CONCLUSION AND FUTURE WORK

This study introduces a robust framework (MMPF) designed to predict job termination outcomes and identify failure types in real time. The framework uses a two-layer strategy: a hybrid soft voting ensemble (DT, KNN, XGBoost, AdaBoost, ANN) to detect failures, followed by an RF model that categorizes the failure types. The framework, designed for cloud-agnostic applications, achieved 99.83% accuracy in failure detection and 99.97% in failure type identification using the Google computing clusters dataset. High precision, recall, and F1-score values highlight the strong performance of the hybrid voting classifier. This performance outperforms previous methods for the prediction of job failure, including those presented in [15], [19A], [20A], [22A], [26A], [36A], and [39A], as shown in Figure 10. In the case of failure type prediction, there are no previous models available for comparison. This approach can contribute to resource optimization and minimize system disruption. By predicting job failures in advance and identifying potential issues at an early stage.

Future research will focus on developing a cloud resource management model that proactively predicts and addresses job failures. This will be achieved through adaptive strategies such as task migration, dynamic resource reallocation, load balancing, and job rescheduling.

ACKNOWLEDGMENT

The authors would like to thank the National Telecommunication Regulatory Authority (NTRA), Egypt, for their support.

REFERENCES

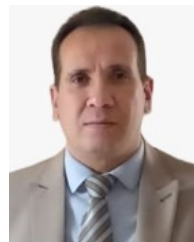
- [1] I. K. Kim, W. Wang, Y. Qi, and M. Humphrey, "Forecasting cloud application workloads with CloudInsight for predictive resource management," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1848–1863, Jul. 2022, doi: [10.1109/TCC.2020.2998017](https://doi.org/10.1109/TCC.2020.2998017).
- [2] D. Saxena, I. Gupta, A. K. Singh, and C.-N. Lee, "A fault tolerant elastic resource management framework toward high availability of cloud services," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 3, pp. 3048–3061, Sep. 2022.
- [3] F. H. Bappy, T. Islam, T. S. Zaman, R. Hasan, and C. Caicedo, "A deep dive into the Google cluster workload traces: Analyzing the application failure characteristics and user behaviors," in *Proc. 10th Int. Conf. Future Internet Things Cloud (FiCloud)*, Aug. 2023, pp. 103–108.
- [4] B. K. Ray, A. Saha, S. Khatua, and S. Roy, "Proactive fault-tolerance technique to enhance reliability of cloud service in cloud federation environment," *IEEE Trans. Cloud Comput.*, vol. 10, no. 2, pp. 1–12, Apr. 2022, doi: [10.1109/TCC.2020.2968522](https://doi.org/10.1109/TCC.2020.2968522).
- [5] M. Ozer, S. Varlioglu, B. Gonen, V. Adewopo, N. Elsayed, and S. Zengin, "Cloud incident response: Challenges and opportunities," in *Proc. Int. Conf. Comput. Sci. Comput. Intell. (CSCI)*, Dec. 2020, pp. 49–54.
- [6] J. Dong, Y. Chen, B. Yao, X. Zhang, and N. Zeng, "A neural network boosting regression model based on XGBoost," *Appl. Soft Comput.*, vol. 125, Aug. 2022, Art. no. 109067, doi: [10.1016/j.asoc.2022.109067](https://doi.org/10.1016/j.asoc.2022.109067).

- [7] X. Chen, L. Yang, Z. Chen, G. Min, X. Zheng, and C. Rong, "Resource allocation with workload-time windows for cloud-based software services: A deep reinforcement learning approach," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1–15, Apr. 2023, doi: [10.1109/TCC.2022.3169157](#).
- [8] L. Luo, S. Meng, X. Qiu, and Y. Dai, "Improving failure tolerance in large-scale cloud computing systems," *IEEE Trans. Rel.*, vol. 68, no. 2, pp. 620–632, Jun. 2019, doi: [10.1109/TR.2019.2901194](#).
- [9] A. Priyadarshini, S. K. Pradhan, S. R. Laha, and D. S. K. Nayak, "Enhancing fault tolerance and load balancing in cloud computing for improved e-healthcare systems performance," in *Proc. 2nd Int. Conf. Ambient Intell. Health Care (ICAHC)*, Nov. 2023, pp. 1–6, doi: [10.1109/icaic59020.2023.10431436](#).
- [10] S. R. Laha, M. Parhi, S. Pattanaik, B. K. Pattanayak, and S. Patnaik, "Issues, challenges and techniques for resource provisioning in computing environment," in *Proc. 2nd Int. Conf. Appl. Mach. Learn. (ICAML)*, Oct. 2020, pp. 157–161, doi: [10.1109/ICAML51583.2020.00039](#).
- [11] S. Choularas and S. Sotiriadis, "An adaptive auto-scaling framework for cloud resource provisioning," *Future Gener. Comput. Syst.*, vol. 148, pp. 173–183, Nov. 2023, doi: [10.1016/j.future.2023.05.017](#).
- [12] S. Abderrazik, M. Hakem, and B. Benmammar, "A distributed fault-tolerant algorithm for load balancing in cloud computing environments," *E3S Web Conf.*, vol. 351, May 2022, Art. no. 01012, doi: [10.1051/e3sconf/202235101012](#).
- [13] C. Meyyappan and C. S. Ravichandran, "Performance analysis of fault-tolerance techniques towards solar fed cascaded multilevel inverter," in *Proc. Int. Conf. Comput. Commun. Informat. (ICCCI)*, Jan. 2021, pp. 1–7, doi: [10.1109/ICCCI50826.2021.9402705](#).
- [14] Y. Xiao, L. Xiao, K. Wan, H. Yang, Y. Zhang, Y. Wu, and Y. Zhang, "Reinforcement learning based energy-efficient collaborative inference for mobile edge computing," *IEEE Trans. Commun.*, vol. 71, no. 2, pp. 864–876, Feb. 2023, doi: [10.1109/TCOMM.2022.3229033](#).
- [15] K. Vani and S. Sujatha, "A machine learning framework for job failure prediction in cloud using hyper parameter tuned MLP," in *Proc. 2nd Int. Conf. Adv. Technol. Intell. Control, Environ., Comput. Commun. Eng. (ICATIECE)*, Dec. 2022, pp. 1–6, doi: [10.1109/ICATIECE56365.2022.10047809](#).
- [16] M. M. Abualhaj, M. M. Al-Tahrawi, and S. N. Al-Khatib, "Performance evaluation of VoIP systems in cloud computing," *J. Eng. Sci. Technol.*, vol. 14, pp. 1398–1405, Apr. 1998.
- [17] M. Kolhar, M. M. Abu-Alhaj, and S. M. A. El-atty, "Cloud data auditing techniques with a focus on privacy and security," *IEEE Secur. Privacy*, vol. 15, no. 1, pp. 42–51, Jan. 2017, doi: [10.1109/MSP.2017.16](#).
- [18] M. M. A. Moni, M. Niloy, M. F.-U.-A. Juboraj, and A. Banik, "AI based cloud failure detection and prevention algorithm," in *Proc. Innov. Power Adv. Comput. Technol. (i-PACT)*, Kuala Lumpur, Malaysia, Dec. 2023, pp. 1–5, doi: [10.1109/i-pact58649.2023.10434295](#).
- [19] Y. Alahmad, T. Daradkeh, and A. Agarwal, "Proactive failure-aware task scheduling framework for cloud computing," *IEEE Access*, vol. 9, pp. 106152–106168, 2021, doi: [10.1109/ACCESS.2021.3101147](#).
- [20] M. Soualhia, F. Khomh, and S. Tahar, "A dynamic and failure-aware task scheduling framework for Hadoop," *IEEE Trans. Cloud Comput.*, vol. 8, no. 2, pp. 553–569, Apr. 2020, doi: [10.1109/TCC.2018.2805812](#).
- [21] M. S. Jassas and Q. H. Mahmoud, "Evaluation of a failure prediction model for large scale cloud applications," in *Advances in Artificial Intelligence*, vol. 12109, C. Goutte and X. Zhu, Eds., Cham, Switzerland: Springer, 2020, pp. 321–327, doi: [10.1007/978-3-030-47358-7_32](#).
- [22] K. Vani and S. Sujatha, "Enhancing fault tolerance system in cloud environment through failure prediction using ensemble learning," in *Proc. 2nd Int. Conf. Intell. Data Commun. Technol. Internet Things (IDCIoT)*, Jan. 2024, pp. 851–858, doi: [10.1109/idciot59759.2024.10467215](#).
- [23] J. Shetty, R. Sajjan, and G. Shobha, "Task resource usage analysis and failure prediction in the cloud," in *Proc. 9th Int. Conf. Cloud Comput., Data Sci. Eng.*, Jan. 2019, pp. 342–348, doi: [10.1109/CONFLUENCE.2019.8776612](#).
- [24] V. Salanke and V. Kowshik, "Task failure prediction and migration in cloud environment," in *Proc. 1st Int. Conf. Commun. Comput. Sci. (InCCCS)*, May 2024, pp. 1–6, doi: [10.1109/InCCCS60947.2024.10593580](#).
- [25] S. Jomah and S. Aji, "Google cloud trace: Characterization of terminated jobs," in *Proc. 2nd Int. Conf. Advancement Comput. Comput. Technol. (InCACCT)*, May 2024, pp. 517–522, doi: [10.1109/inccact61598.2024.10551146](#).
- [26] A.-I. Tuns and A. Spataru, "Cloud service failure prediction on Google's Borg cluster traces using traditional machine learning," in *Proc. 25th Int. Symp. Symbolic Numeric Algorithms Sci. Comput. (SYNASC)*, Sep. 2023, pp. 162–169, doi: [10.1109/synasc61333.2023.00029](#).
- [27] A. Ulah, Z. Alomari, S. Alkhashayni, and D. Al-zaleq, "Improvement in task allocation for VM and reduction of Makespan in IaaS model for cloud computing," *Cluster Comput.*, vol. 27, pp. 11407–11426, Jul. 2024, doi: [10.1007/s10586-024-04539-8](#).
- [28] C. Liu, L. Dai, Y. Lai, G. Lai, and W. Mao, "Failure prediction of tasks in the cloud at an earlier stage: A solution based on domain information mining," *Computing*, vol. 102, no. 9, pp. 2001–2023, Sep. 2020, doi: [10.1007/s00607-020-00800-1](#).
- [29] J. Gao, H. Wang, and H. Shen, "Task failure prediction in cloud data centers using deep learning," *IEEE Trans. Services Comput.*, vol. 15, no. 3, pp. 1411–1422, May 11, 2022, doi: [10.1109/TSC.2020.2993728](#).
- [30] A. Rosa, L. Y. Chen, and W. Binder, "Failure analysis and prediction for big-data systems," *IEEE Trans. Services Comput.*, vol. 10, no. 6, pp. 984–998, Nov. 2017, doi: [10.1109/TSC.2016.2543718](#).
- [31] T. Islam and D. Manivannan, "Predicting application failure in cloud: A machine learning approach," in *Proc. IEEE Int. Conf. Cognit. Comput. (ICCC)*, Jun. 2017, pp. 24–31, doi: [10.1109/IEEEICCC.2017.11](#).
- [32] C. Liu, J. Han, Y. Shang, C. Liu, B. Cheng, and J. Chen, "Predicting of job failure in compute cloud based on online extreme learning machine: A comparative study," *IEEE Access*, vol. 5, pp. 9359–9368, 2017, doi: [10.1109/ACCESS.2017.2706740](#).
- [33] X. Chen, C.-D. Lu, and K. Pattabiraman, "Failure prediction of jobs in compute clouds: A Google cluster case study," in *Proc. IEEE Int. Symp. Softw. Rel. Eng. Workshops*, Nov. 2014, pp. 341–346, doi: [10.1109/ISSREW.2014.105](#).
- [34] N. El-Sayed, H. Zhu, and B. Schroeder, "Learning from failure across multiple clusters: A trace-driven approach to understanding, predicting, and mitigating job terminations," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jun. 2017, pp. 1333–1344, doi: [10.1109/ICDCS.2017.317](#).
- [35] T. Hongyan, L. Ying, W. Long, G. Jing, and W. Zhonghai, "Predicting misconfiguration-induced unsuccessful executions of jobs in big data system," in *Proc. IEEE 41st Annu. Comput. Softw. Appl. Conf. (COMPSAC)*, vol. 1, Jul. 2017, pp. 772–777, doi: [10.1109/COMPSAC.2017.191](#).
- [36] F. Bashir and F. Z. Khan, "Analysis of job failure prediction in a cloud environment by applying machine learning techniques," *Int. J. Innov. Sci. Technol.*, vol. 4, no. 4, pp. 1–10, Oct. 2022.
- [37] B. Mohammed, I. Awan, H. Ugail, and M. Younas, "Failure prediction using machine learning in a virtualised HPC system and application," *Cluster Comput.*, vol. 22, no. 2, pp. 471–485, Jun. 2019, doi: [10.1007/s10586-019-02917-1](#).
- [38] M. S. Jassas and Q. H. Mahmoud, "Analysis of job failure and prediction model for cloud computing using machine learning," *Sensors*, vol. 22, no. 5, p. 2035, Mar. 2022, doi: [10.3390/s22052035](#).
- [39] S. Divesh and Y. Samarawickrama, "Improving fault tolerance of a task in cloud using ensemble approach," M.S. thesis, School Comput., Nat. College Ireland, Dublin, Ireland. [Online]. Available: <https://norma.ncirl.ie/id/eprint/7107>
- [40] A. Marahatta, Q. Xin, C. Chi, F. Zhang, and Z. Liu, "PEFS: AI-driven prediction based energy-aware fault-tolerant scheduling scheme for cloud data center," *IEEE Trans. Sustain. Comput.*, vol. 6, no. 4, pp. 655–666, Oct. 2021, doi: [10.1109/TSUSC.2020.3015559](#).
- [41] R. R. Swain, D. S. K. Nayak, and T. Swarnkar, "A comparative analysis of machine learning models for colon cancer classification," in *Proc. Int. Conf. Adv. Power, Signal, Inf. Technol. (APSIT)*, Jun. 2023, pp. 1–5, doi: [10.1109/apsit58554.2023.10201691](#).
- [42] M. S. Al-Asaly, M. A. Bencherif, A. Alsanad, and M. M. Hassan, "A deep learning-based resource usage prediction model for resource provisioning in an autonomic cloud computing environment," *Neural Comput. Appl.*, vol. 34, no. 13, pp. 10211–10228, Jul. 2022, doi: [10.1007/s00521-021-06665-5](#).
- [43] M. S. Kumar, A. Choudhary, I. Gupta, and P. K. Jana, "An efficient resource provisioning algorithm for workflow execution in cloud platform," *Cluster Comput.*, vol. 25, no. 6, pp. 4233–4255, Dec. 2022, doi: [10.1007/s10586-022-03648-6](#).
- [44] M. Sohani and S. C. Jain, "A predictive priority-based dynamic resource provisioning scheme with load balancing in heterogeneous cloud computing," *IEEE Access*, vol. 9, pp. 62653–62664, 2021, doi: [10.1109/ACCESS.2021.3074833](#).

- [45] Google. *Google 2019 Cluster Sample*. Accessed: Oct. 5, 2024. [Online]. Available: <https://www.kaggle.com/datasets/derrick-mwiti/google-2019-cluster-sample>
- [46] S. Ouhamme, Y. Hadi, and A. Ullah, "An efficient forecasting approach for resource utilization in cloud data center using CNN-LSTM model," *Neural Comput. Appl.*, vol. 33, no. 16, pp. 10043–10055, Mar. 2021, doi: [10.1007/s00521-021-05770-9](https://doi.org/10.1007/s00521-021-05770-9).
- [47] X. Gao, R. Liu, A. Kaushik, and H. Zhang, "Dynamic resource allocation for virtual network function placement in satellite edge clouds," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2252–2265, Jul. 2022, doi: [10.1109/TNSE.2022.3159796](https://doi.org/10.1109/TNSE.2022.3159796).
- [48] M. Tirmazi, A. Barker, N. Deng, M. Haque, Z. Qin, S. Hand, M. Harchol-Balter, and J. Wilkes, "Borg: The next generation," in *Proc. 15th Eur. Conf. Comput. Syst. (EuroSys)*, Heraklion, Greece, Apr. 2020, no. 30, pp. 1–14, doi: [10.1145/3342195.3387517](https://doi.org/10.1145/3342195.3387517).
- [49] G. Amvrosiadis, J. W. Park, G. R. Ganger, G. A. Gibson, E. Baseman, and N. DeBardeleben, "On the diversity of cluster workloads and its impact on research results," in *Proc. USENIX Annu. Tech. Conf.*, 2018, pp. 533–546.
- [50] M. Soualhia, F. Khomh, and S. Tahar, "Predicting scheduling failures in the cloud: A case study with Google clusters and Hadoop on Amazon EMR," in *Proc. IEEE 17th Int. Conf. High Perform. Comput. Commun., 7th Int. Symp. Cyberspace Saf. Secur., IEEE 12th Int. Conf. Embedded Softw. Syst.*, Aug. 2015, pp. 58–65, doi: [10.1109/HPCC-CSS-ICSS.2015.170](https://doi.org/10.1109/HPCC-CSS-ICSS.2015.170).
- [51] T. Islam and D. Manivannan, "FaCS: Toward a fault-tolerant cloud scheduler leveraging long short-term memory network," in *Proc. 6th IEEE Int. Conf. Cyber Secur. Cloud Comput. (CSCloud)/ 5th IEEE Int. Conf. Edge Comput. Scalable Cloud (EdgeCom)*, Jun. 2019, pp. 1–6, doi: [10.1109/CSCLOUD/EDGECOM.2019.00010](https://doi.org/10.1109/CSCLOUD/EDGECOM.2019.00010).
- [52] Z. Li, L. Liu, and D. Kong, "Virtual machine failure prediction method based on AdaBoost-hidden Markov model," in *Proc. Int. Conf. Intell. Transp., Big Data Smart City (ICITBS)*, Jan. 2019, pp. 700–703, doi: [10.1109/ICITBS.2019.00173](https://doi.org/10.1109/ICITBS.2019.00173).
- [53] D. S. K. Nayak, S. P. Routray, S. Sahoo, S. K. Sahoo, and T. Swarnkar, "A comparative study using next generation sequencing data and machine learning approach for Crohn's disease (CD) identification," in *Proc. Int. Conf. Mach. Learn., Comput. Syst. Secur. (MLCSS)*, Aug. 2022, pp. 17–21, doi: [10.1109/mlcss57186.2022.00012](https://doi.org/10.1109/mlcss57186.2022.00012).
- [54] D. Y. Omkari and K. Shaik, "An integrated two-layered voting (TLV) framework for coronary artery disease prediction using machine learning classifiers," *IEEE Access*, vol. 12, pp. 56275–56290, 2024, doi: [10.1109/ACCESS.2024.3389707](https://doi.org/10.1109/ACCESS.2024.3389707).
- [55] X. Xu, R. Mo, F. Dai, W. Lin, S. Wan, and W. Dou, "Dynamic resource provisioning with fault tolerance for data-intensive meteorological workflows in cloud," *IEEE Trans. Ind. Informat.*, vol. 16, no. 9, pp. 6172–6181, Sep. 2020, doi: [10.1109/TII.2019.2959258](https://doi.org/10.1109/TII.2019.2959258).
- [56] P. Kumari and P. Kaur, "A survey of fault tolerance in cloud computing," *J. King Saud Univ.*, vol. 33, no. 10, pp. 1159–1176, 2018, doi: [10.1016/j.jksuci.2018.09.021](https://doi.org/10.1016/j.jksuci.2018.09.021).
- [57] P. Padmakumari and A. Umamakeswari, "Task failure prediction using combine bagging ensemble (CBE) classification in cloud workflow," *Wireless Pers. Commun.*, vol. 107, no. 1, pp. 23–40, Jul. 2019, doi: [10.1007/s11277-019-06238-9](https://doi.org/10.1007/s11277-019-06238-9).
- [58] A. Rosa, L. Y. Chen, and W. Binder, "Predicting and mitigating jobs failures in big data clusters," in *Proc. 15th IEEE/ACM Int. Symp. Cluster, Cloud Grid Comput.*, May 2015, pp. 221–230, doi: [10.1109/CCGRID.2015.139](https://doi.org/10.1109/CCGRID.2015.139).
- [59] U. K. Jena, P. K. Das, and M. R. Kabat, "Hybridization of meta-heuristic algorithm for load balancing in cloud computing environment," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 6, pp. 2332–2342, Jun. 2022, doi: [10.1016/j.jksuci.2020.01.012](https://doi.org/10.1016/j.jksuci.2020.01.012).
- [60] H. Sun, H. Yu, G. Fan, and L. Chen, "QoS-aware task placement with fault-tolerance in the edge-cloud," *IEEE Access*, vol. 8, pp. 77987–78003, 2020, doi: [10.1109/ACCESS.2020.2977089](https://doi.org/10.1109/ACCESS.2020.2977089).
- [61] A. Arabiat, M. Hassan, and O. Almomani, "WEKA-based machine learning for traffic congestion prediction in amman city," *IAES Int. J. Artif. Intell. (IJ-AI)*, vol. 13, no. 4, p. 4422, Dec. 2024.



AHMED ELKARADAWY was born in Kafr El Sheikh, Egypt. He received the bachelor's degree in systems and computer engineering, in 2007, and the M.Sc. degree in systems and computer engineering from Al-Azhar University, Nasr City, Cairo, Egypt, in 2017, where he is currently pursuing the Ph.D. degree in systems and computer engineering. He is the Acting Director of the Wireless Terrestrial Service Department, National Telecommunications Regulatory Authority (NTRA), Egypt, and the Vice Chair of the ITU-R Study Group 6. His research interests include cloud computing, machine learning, and wireless networks.



AYMAN ELSHENAWY received the Ph.D. degree in systems and computer engineering from Al-Azhar University, Egypt, in 2013. He is currently with the Department of Software Engineering, Faculty of Information Technology, Al-Ahliyya Amman University, Amman, Jordan. He is also a former Associate Professor at the Software Engineering and Information Technology Department, Faculty of Engineering and Technology, Egyptian Chinese University. He also works as an Associate Professor at the Systems and Computer Engineering Department, Faculty of Engineering, Al-Azhar University. His research interests include cloud robotics, educational machine learning, deep learning applications, and NLP. He made significant contributions to the stated research fields, publishing in top-quality journals/conferences, including *The Computer Journal*, *International Journal of Imaging and Robotics*, *Wireless Personal Communications*, *Education and Information Technologies*, *International Journal of Advanced Computer Science and Applications*, *Journal of Information Technology Management*, *Intelligent Automation and Soft Computing*, *International Journal of Information Technology* (Singapore), and *Scientific Reports*. Also, he is a participant in some IEEE conferences, such as the International Review of Automatic Control (IREACO).



HANY HARB received the B.Sc. degree in computers and control engineering from the Faculty of Engineering, Ain Shams University, Egypt, in 1978, the M.Sc. degree in computers and systems engineering from the Faculty of Engineering, Al-Azhar University, Egypt, in 1981, and the Ph.D. degree in computer science and the M.Sc. degree in operations research (MSOR) from the Institute of Technology (IIT), USA, in 1986 and 1987, respectively. He is currently a Professor of software engineering with the System Engineering Department, Faculty of Engineering, Al-Azhar University. His research interests include artificial intelligence, cloud computing, and distributed systems.

• • •